



Spatial Audio Navigation Device for Visually Impaired

a dissertation submitted for the degree of Electrical and Electronic Engineering/Electronic
and Computer Engineering

Isaac Allinson

230333675

May 2026

Word count: 5971

Self-reflection and feedback

Complete this section so that your examiners can provide tailored feedback to your work (up to 50 words per answer). This is a reflective narrative for feedback purposes only, your marks will not be informed by information on this page.

1) I believe a strong part of my submission is:

The Literature review section, which does a deep review of existing technologies and gives a clear reasoning for this project

2) An area I struggled with is:

The testing section. I found it difficult to know what to test other than latency and accuracy of the device, although I do recognise a larger participant group would've strengthened my results

3) I would particularly like feedback/help with:

Any additional tests that could've been done to evaluate the system and whether the objectives laid out were appropriate in section 2.

Engineering skills portfolio indexing

Your dissertation needs to demonstrate you have achieved certain learning outcomes as a result of your work which you must reflect on. Please use this checklist to index the section number(s) which demonstrate the appropriate skill. This page **must be filled out** and included in the final report to pass the project.

Engineering skill	Page(s) where demonstrated
Environmental and societal impact Where in your report have you demonstrated that you have considered the wider impact on society to the broader area that you have been working in? For example, if your project linked to reduction of greenhouse gasses or personal mobility, how does your work fit into these broader societal goals?	5,6
Ethical considerations All projects contain ethical considerations, although some are more obvious than others. Have you researched an potential negative impacts of your project? Are there ethical questions in literature or journalism about the area you are working in? What is the general consensus of the engineering community of these issues?	5,6
Risk management and mitigation For example, have you planned for unexpected outcomes or circumstances? How did you manage changes to your project? Did you decide to change your project and if so how did it change the risk profile of the work? This is usually achieved through regular review, reflection and modification of your project plan, but you must demonstrate this.	9,22
Intellectual property security and data management Whilst some sectors have a more obvious security aspect, all projects require a degree of data protection and management. How have you ensured that the data or intellectual property (IP) which you have generated within your project is secure from external actors or accidental loss? Have you used backups or password protected file stores? Where have you demonstrated an awareness of how IP is captured. How was information management included in part of your project planning?	16
Equality, diversity and inclusion Linking to your considerations of societal impact, where have you considered the inclusivity of your work? Have you demonstrated that you have considered how your work may impact different groups and made efforts to minimise this? Is your work accessible to a wide range of users and could others from a different background to yourself repeat your results based on what you have written in your report?	4, 18, 21
Quality of information Where in your report have you discussed the <i>quality</i> of the information you have researched or the data you have presented? Have you reflected on how this could be improved?	5,6,7, 21
Continual professional development and self-reflection Where in your report have you reflected on your own performance throughout the project? Have you identified personal strengths and achievements as well as weaknesses? Have you made plans for exploiting the strengths and improving the weaknesses?	22, 23

Table of Contents

1. Problem Definition and Context	5
2. Aims and Objectives	5
3. Literature Review.....	6
3.1 Visual Impairment and the Challenge of Independent Navigation	6
3.2 Existing Assistive Navigation Technologies.....	7
3.3 Spatial Audio and Head-Related Transfer Functions	7
3.4 Head Tracking and Gyroscopic Sensing	8
3.5 Embedded Platforms for Assistive Audio Devices	9
3.6 Summary and Research Gap	9
Chapter 4: Implementation.....	10
4.1 Overview.....	10
4.2 Hardware Selection and Bill of Materials	10
4.3 Physical Assembly and Wiring	11
4.4 Software Stack.....	16
4.5 IMU Driver and Orientation Estimation	17
4.6 HRTF Convolution Engine.....	18
4.7 Flask Web Interface and Calibration	18
4.8 Summary	19
5. Solution Testing and Design Iterations	20
5.1 Test Strategy	20
5.2 IMU and Orientation Pipeline	20
5.3 Audio Pipeline Latency	21
5.4 Spatialisation Accuracy.....	22
5.5 Head-Trackled Spatial Audio.....	23
5.6 Design Iterations	23
6. Conclusion.....	24
References.....	24
Appendix A – Code Listings.....	26
A.1 – app.py	26
A.2 – spatial_engine.py.....	28
A.3 – sofa_reader.py.....	32
A.4 – imu_driver.py.....	33

1. Problem Definition and Context

Navigation remains one of the most significant challenges for visually impaired people. Existing aids, such as the white cane or GPS based apps, each have well documented limitations, particularly indoors where satellite signal is unavailable and spatial awareness is necessary. These are discussed in detail in Section 3, but the common issue is that there is no existing low cost, wearable solution that gives a visually impaired user real time directional guidance indoors. When vision becomes impaired, hearing becomes the primary sense for spatial awareness, which makes audio a natural option for navigation aids.

This project sets out to address that gap by building a wearable device that uses spatial audio to convey direction. The device renders a virtual sound source at a user defined position and updates in real time as the user rotates their head, so the sound appears fixed at one point in the room. The user can then orient themselves towards the sound, which could be placed on a doorway or object, simply by turning their head.

The project was to be kept under a budget of £50 as many assistive products are expensive and inaccessible to the people who need them most. Keeping the cost low was a large part of the design and directly impacted the hardware choices made throughout the project.

The device relies on a target position being set through the web interface, either by the user or with assistance, and then the audio guides them to orient towards it. This is a meaningful and useful capability on its own, and a necessary foundation for any future system that adds automatic object detection. An extension would be to combine the audio layer with a depth camera or ultrasonic sensor which can detect objects.

2. Aims and Objectives

The aim of this project was to design, build and evaluate a low-cost wearable device capable of providing real time spatial audio guidance to visually impaired users for indoor navigation.

To achieve this, the following objectives were set:

1. Use a compact embedded processing platform capable of running a real time HRTF convolution pipeline whilst remaining suitable for head mounting in terms of size and weight
2. Implement a head orientation estimate using an IMU, producing a stable and drift corrected orientation estimate in real time

3. Implement a HRTF based spatial audio engine capable of producing a virtual sound source at a user defined position which updates constantly depending on head orientation
4. Provide an easy way to control the device without mouse and keyboard, which is accessible from any device
5. Keep total cost of components within £50

The measurable outcomes from these objectives are set out in table 2.1.

Specification	Target
Latency	< 80 ms
Orientation Accuracy	< 5° Yaw error
Spatialisation Accuracy	> 75% correct response rate for all azimuths
Total Component Cost	< £50
Total Component Weight	< 100g
Self-Contained System	No external hardware needed

The 80ms latency target is taken from the threshold below which head tracking delay becomes unnoticeable to listeners, as discussed in Section 5.3. The 5° orientation accuracy reflects the angular resolution of the CIPIC dataset, where exceeding this would cause incorrect HRTF selection. The 75% spatialisation accuracy target was set to be a reasonable baseline for a non-personalised HRTF system.

3. Literature Review

3.1 Visual Impairment and the Challenge of Independent Navigation

Vision loss remains as one of the most serious obstacles to independent movement and everyday living. The World Health Organization estimates that 2.2 billion people worldwide live with some form of vision impairment [1]. Sight accounts for roughly 80% of the sensory information humans use to make sense of their environment [2] and the loss or reduction of this sense greatly impacts the ability to perceive and safely interact with the surrounding environment. For those with severe or complete vision loss, the ability to get around independently, especially in unfamiliar environments, has a direct impact on their quality of life. Orientation and mobility specialists have recognised that effective navigation requires the ability to use multiple sensory channels, among which auditory perception becomes the primary means of spatial awareness when vision is impaired [3].

The psychological and social effects of restricted mobility are well documented. Research has found that people with visual impairments are more likely to experience social isolation and

withdraw from everyday activities, with anxiety around independent navigation being a key contributing factor [4]. This has driven significant research into assistive technologies that can deliver spatial information through other senses, particularly hearing and touch.

3.2 Existing Assistive Navigation Technologies

The white cane remains the most widely adopted mobility aid for visually impaired individuals worldwide. While it is cheap, simple and gives useful feedback about obstacles at ground level, it cannot detect objects at head height and gives no sense of the wider layout of the room [5]. Electronic travel aids have been developed, which use ultrasonic sensors or other technology to detect nearby obstacles and warn the user through vibration or audio cues. [6]. The WeWalk smart cane has improved this by using a built in ultrasonic sensor along with the ability to connect to a smartphone, which allows it to pick up chest and head level obstacles within a 0.8-2.5m range [7]. However, these devices provide only coarse proximity data rather than precise directional information and the WeWALK's dependency on additional hardware purchase has been noted as a barrier to accessibility [7].

GPS-based navigation systems, including smartphone applications such as BlindSquare and Google Maps with voice guidance, have considerably expanded outdoor navigation capabilities for visually impaired users [8]. However, both rely heavily on satellite signal and accurate mapping data, which makes them unreliable indoors. Kandalan and Namuduri [8] reviewed indoor navigation techniques for visually impaired users and found that GPS on mobile devices typically only achieves metre level accuracy inside buildings, which is nowhere near precise enough to safely navigate walls and furniture. As Akin and Cömert [9] had the same problem with their smartphone based augmented reality navigation app. Despite achieving solid object detection performance with an RMSE of 1.107m at ranges beyond 20m, the system was limited to outdoor use because of the unreliability of GPS indoors. This shows a clear gap between indoor navigation and object localisation.

Some newer computer vision approaches integrate cameras with real time object recognition algorithms. Systems using deep learning models identify and describe objects in a scene [6]. Although promising, these approaches are computationally intensive and convey spatial information primarily through speech descriptions, which can be cognitively demanding in dynamic environments. Akin and Cömert [9] acknowledge that spoken feedback describing redundant or inappropriate objects can cause confusion for visually impaired users. This highlights a fundamental limitation of text to speech based spatial feedback, and suggests that the use of an alternative output that conveys directional information more rapidly could be more practical.

3.3 Spatial Audio and Head-Related Transfer Functions

Spatial audio refers to the reproduction of sound in a manner that conveys the three-dimensional position of a source relative to the listener. The human auditory system localises sound sources using a combination of interaural time difference (ITD), interaural level difference (ILD) and spectral shaping introduced by the pinnae, head and torso [10]. These spectral transformations are collectively characterised by the Head-Related Transfer Function (HRTF), a frequency domain representation of how sound from a given direction is filtered before reaching each eardrum. By convolving an audio signal with a pair of HRTFs corresponding to a desired source direction, it is possible to present the signal over so that it is

perceived as coming from that direction in space, which is a technique known as binaural synthesis [11].

A widely used reference collection of HRTF measurements is the CIPIC HRTF Database, produced by the Center for Image Processing and Integrated Computing at the University of California, Davis [12]. The database provides Head-Related Impulse Responses (HRIRs), which is the time-domain equivalent of HRTFs, measured at 1,250 spatial positions across 45 subjects, using a manikin to serve as a generalised human head and torso. The CIPIC dataset is distributed in the AES69 SOFA (Spatially Oriented Format for Acoustics) file format, a standardised container for storing acoustic measurements alongside their associated source and receiver positions [13]. SOFA is the current international standard for HRTF data interchange and is supported by open-source libraries including pysofaconventions, which enables straightforward loading and indexing of HRTF datasets in Python. The use of the CIPIC KEMAR dataset is a practical choice for a low-cost device where individualised HRTF measurement is not practical.

The potential of HRTF-based spatial audio as a navigation aid for visually impaired users has been demonstrated in prior research. Ribeiro et al. [14] developed a head-mounted auditory augmented reality device that sonifies real-world objects at their corresponding spatial coordinates using HRTFs from the CIPIC database, combined with a 3D gyroscope for head tracking. Their system conveyed distance by reducing the source volume by 6 dB for each doubling in distance, combined with a fixed reverberation component, using the relationship between sound intensity and reverb level as cues for how far away a source is. [14]. In a user study with 14 participants, 9 correctly identified at least 5 of 6 test scenes, demonstrating that users can effectively build mental maps from spatialised acoustic signals even without personalised HRTFs or prior training [14]. Ribeiro et al explicitly demonstrate that combining spatial audio with gyroscope-based head tracking on a wearable embedded platform is technically feasible and perceptually effective, demonstrating the approach taken in this project is feasible.

A key challenge in HRTF-based systems is variability as HRTFs are highly specific to each person due to anatomical differences in ear and head geometry. Using a non-individualised HRTF, as is practical in a low cost device, can degrade the accuracy of the localisation, especially for the front and back [15]. Ribeiro et al [14] acknowledge that some participants in their study reported difficulty localising spatialised sounds, attributing this partly to mismatch between their personal HRTFs and the KEMAR HRTF used and suggest that a practical device would benefit from personalised HRTF measurement. That said, several studies have shown that non-individualised HRTFs are still good enough for reliable left-right directional guidance, particularly when combined with head tracking which provides additional localisation cues through motion. [16].

3.4 Head Tracking and Gyroscopic Sensing

A key improvement over static binaural rendering is updating the audio in real time based on head movement. When a listener turns their head towards a real sound source, the perceived direction shifts naturally, which provides strong localisation cues and helps resolve front and back confusion, which is one of the main weaknesses of static HRTF rendering. [17]. Dynamic binaural synthesis, in which the HRTF selection is updated in real time based on measured head orientation, closely replicates this. Ribeiro et al [14] implement this by sampling a 3D

gyroscope at 40 Hz and updating a rotation matrix iteratively, ensuring that virtual audio sources remain anchored to their real-world coordinates as the user moves their head.

Micro-electromechanical systems (MEMS) gyroscopes, such as those found in the MPU-6050 inertial measurement units (IMUs), are compact, low power and low cost, making them suitable for wearable assistive devices [18]. In practice, gyroscopes alone are subject to drift, which is a result of gradual accumulation of integration error that causes the estimated orientation to diverge over time. This is typically reduced through algorithms such as the Madgwick filter [19], which combines gyroscope and accelerometer data using a gradient descent algorithm to produce a stable, drift corrected orientation estimate. The filter outputs a quaternion, which is a four number representation of 3D rotation, which is then converted to Euler angles (yaw, pitch and roll) that can be used to select the correct HRTF pair for a given head position. Ribeiro et al. [14] also use an accelerometer alongside the gyroscope, using it to estimate the gravity vector and work out which way is down relative to the device.

3.5 Embedded Platforms for Assistive Audio Devices

Choosing the right embedded platform was an important decision for this project. It needed enough processing power to handle real time HRTF convolution whilst being power efficient, small and cheap enough for a wearable device. The Raspberry Pi family of single board computers is a common choice in academic and prototyping work due to its combination of a multi-core ARM processor, GPIO interface, Linux based software ecosystem and low cost. [20]. The Raspberry Pi 3 Model B, for instance, features a quad-core 64-bit ARM Cortex-A53 processor running at 1.2 GHz and supports audio output via USB or HAT-based interfaces, making it a common choice for audio prototyping work.

For a wearable device where size and weight matter, a more compact platform was needed. The Raspberry Pi Zero 2 W is significantly smaller at 65×30 mm and around 10 g [21], while retaining the same processor and built in Wi-Fi. This means it can run the same Python processing pipeline as larger models whilst being much better suited to head mounting. The Zero 2 W does not have a dedicated audio jack, so an external USB DAC is needed for audio playback. This is not really a limitation though as a USB DAC provides clean stereo output and runs at the 48 kHz sample rate required by the CIPIC dataset.

Real time HRTF convolution works by processing audio in blocks using FIR filtering. For typical HRTF filter lengths of 128 to 512 taps per channel, the computational load is manageable on modern ARM processors using FFT based convolution, as implemented in libraries such as FFTW [22]. Ribeiro et al. [14] reported a convolution latency of just 10 ms using an FFT accelerated engine, confirming that real time HRTF filtering is achievable on modest embedded hardware. The Raspberry Pi also has I2C buses which make it straightforward to connect IMU modules for the head tracking pipeline.

3.6 Summary and Research Gap

The literature makes a clear case for the system proposed in this project. Existing assistive navigation technologies each tackle part of the problem but none solve it completely. White canes and ultrasonic aids lack directional precision, GPS fails indoors [8], and vision-based systems like from Akin and Cömert [9] are computationally demanding and limited to

outdoor use. The closest existing work is Ribeiro et al. [14], who built a head mounted auditory AR device combining HRTF spatialisation, gyroscopic head tracking and a camera and showed that users could build spatial mental maps from the audio alone. However, their prototype relied on a tethered workstation for processing and used a Microsoft Kinect, making it impractical for everyday use. This project addresses those limitations directly by implementing the same spatial audio guidance capability on a self-contained, low-cost Raspberry Pi Zero 2 W using a standard IMU, with indoor navigation as the primary use case.

Chapter 4: Implementation

4.1 Overview

This chapter describes the physical construction and software implementation of the auditory navigation device. The system is designed to assist visually impaired users in locating objects and navigating indoor environments through audio cues synthesised using Head-Related Transfer Functions (HRTFs). The implementation consists of four components: a wearable headband supporting the components, a Raspberry Pi Zero 2 W being used as the processing unit, an MPU 6050 to provide orientation data and a Flask web interface enabling runtime control and calibration.

Rather than building the full system at once, each hardware and software component was developed and tested in isolation. This allowed faults to be identified and localised and prevented compounding errors. Several design changes arose from this process: the selection of the processing platform is discussed in Section 4.2, the wiring and physical assembly in Section 4.3, operating system and configuration in Section 4.4 and the software pipeline described through Section 4.5 to 4.9 and Section 4.10 provides a chapter summary and links to the testing and evaluation described in chapter 5.

4.2 Hardware Selection and Bill of Materials

Hardware selection was driven by the need for a compact, lightweight platform capable of real time FFT convolution and low enough power consumption for portable use. The Raspberry Pi 3 Model B was initially considered because of its widespread use and relatively strong processing performance. However, at 85×56 mm and 45g, it is too large and heavy for comfortable head mounting. A Raspberry Pi Zero 2 W was therefore chosen instead. As established in Section 3.5, given its significantly reduced size, weight and a quad core ARM Cortex-A53 processor, it is well suited to a wearable audio processing application, and its integrated wireless LAN enables SSH access without additional hardware.

For the IMU, an MPU 6050 was selected. It has a three-axis accelerometer and three axis gyroscope on a single chip, communicates over I2C and is available on a compact breakout board (approximately 20 x 16 mm). Having both sensors on one chip matters because the Madgwick filter needs both to work properly.

Due to the lack of an audio jack on the Raspberry Pi Zero 2 W, HDMI audio output was initially considered but not chosen as it requires a display connection and introduces unnecessary complexity for a wearable device. Instead, a USB DAC dongle was selected to convert the digital audio stream into an analogue stereo signal suitable for sending audio to

headphones, connected to the pi by a Micro USB OTG adapter. This provides a stereo analogue output at 48kHz which is compatible with the CIPIC HRTF dataset sample rate. The complete bill of materials is shown in Table 4.1.

Component	Model / Specification	Purpose	Cost
Processing unit	Raspberry Pi Zero 2 W	Runs software + Wi-Fi for SSH access	£25
IMU	MPU 6050	Head orientation via accelerometer + gyroscope	£4
Audio output	USB DAC dongle	Converts digital audio to analogue stereo output at 48kHz 16 bit	£3
Headphones	Turtle Beach Headphones	Stereo playback for binaural audio	£10
Power supply	USB power bank (5 V)	Enables the device to be portable	£10
Headband	Elastic sports headband	Mounting platform for IMU and wiring	£1
USB Adapter	Micro USB OTG adapter with USB A port	Connects USB audio adapter to Pi Zero 2 W	£2

Table 4.1: Bill of materials for the audio navigation device prototype.

4.3 Physical Assembly and Wiring

The prototype housing consists of two separate enclosures, each designed in Onshape and 3D printed in PLA. Each case holds a single component and mounts directly onto the elastic headband via slots on either side of the enclosure through which the headband passes, securing it firmly without adhesive. Holes were incorporated into each case to allow the connecting wires to pass through cleanly.

The Raspberry Pi Zero 2 W enclosure was positioned towards the rear of the headband, distributing the weight of the two components more evenly and keeping cable runs reasonably short. The enclosure design is shown in Figure 4.1 and Figure 4.2.

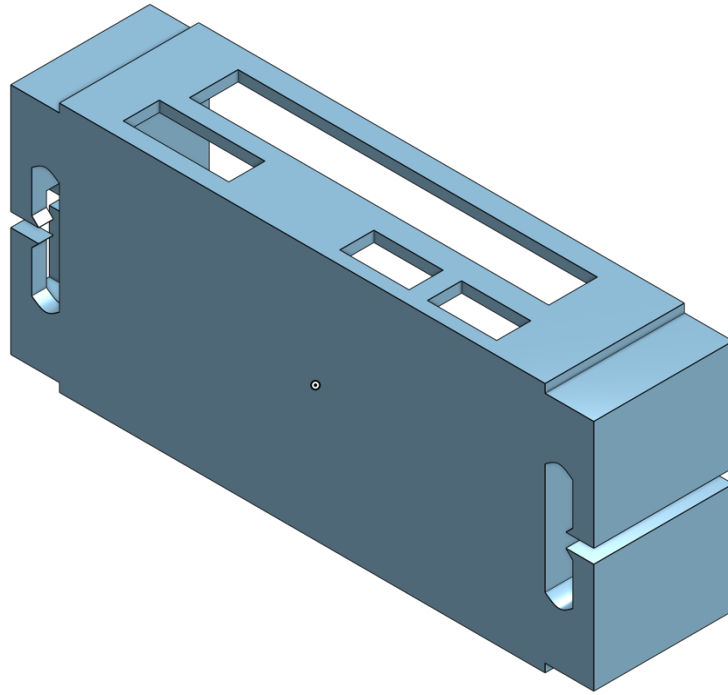


Figure 4.1: Onshape CAD model of the Raspberry Pi Zero 2 W enclosure isometric view.

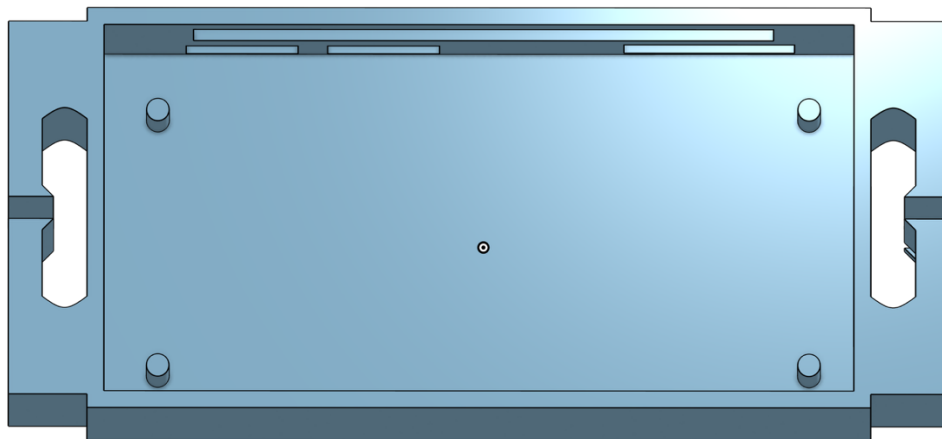


Figure 4.2: Onshape CAD model of Raspberry Pi enclosure top view.

The IMU enclosure was positioned at the front centre of the headband. Placing the MPU-6050 here ensures the sensor tracks head rotation, instead of the user's full body rotation which enables the device to know which way the user is facing. The enclosure design is shown in Figure 4.3 and 4.4.

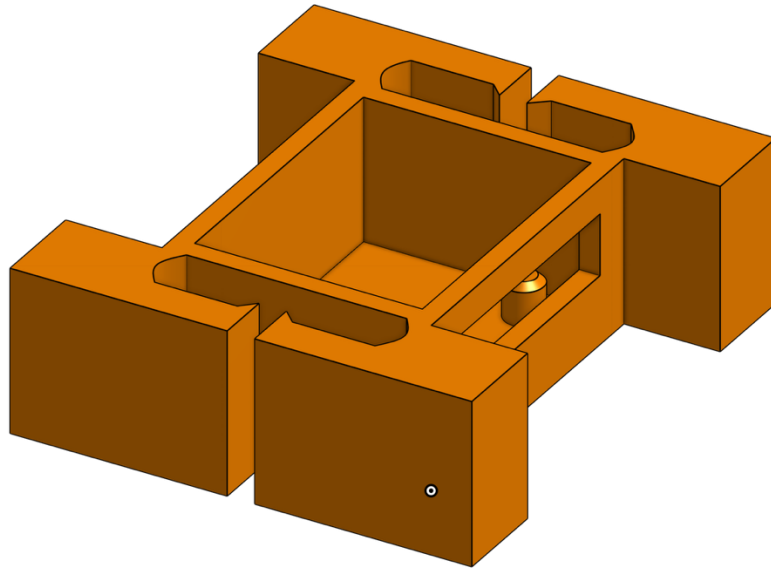


Figure 4.3: Onshape CAD model of the MPU-6050 enclosure isometric view.

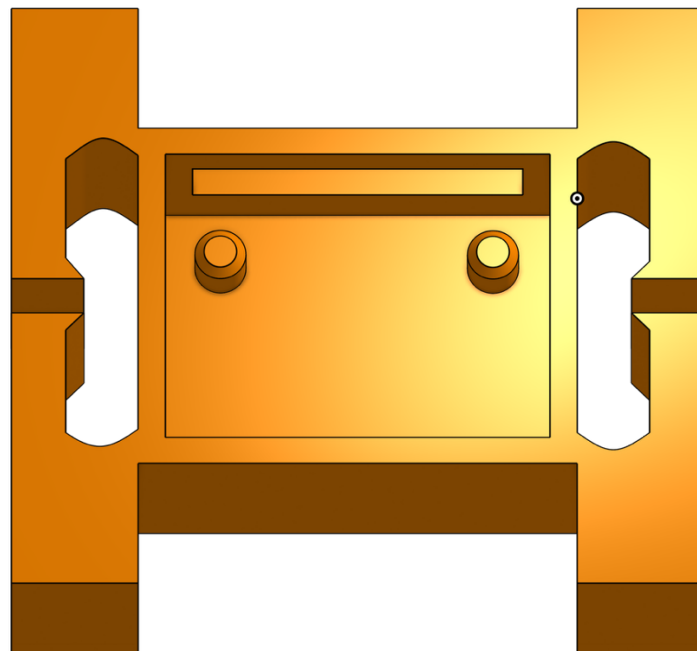


Figure 4.4: Onshape CAD model of the MPU-6050 enclosure top view.

The I2c connections between the MPU-6050 and the Raspberry Pi were made using jumper wires going through the holes in each enclosure. Whilst jumper wires are straightforward to connect and reconfigure, their connectors can introduce some contact failures if the headband

flexes too much. Replacing them with soldered stranded wires would be a good improvement in a future iteration. The four I2C connections made are shown in Table 4.2.

MPU-6050 Pin	Raspberry Pi Pin	GPIO
VCC	Pin 1	3.3 V
GND	Pin 6	Ground
SDA	Pin 3	GPIO 2
SCL	Pin 5	GPIO 3

Table 4.2: I²C wiring connections between the MPU-6050 and Raspberry Pi Zero 2 W.

As GPIO 2 and GPIO 3 already have internal pull up resistors, no external pull ups were needed.

The USB audio adapter was connected to the Pi via the micro-USB OTG adapter. Power is supplied separately through the Pi's micro-USB power port from the battery bank. The headphones were plugged into the 3.5mm jack on the adapter. The front of the device can be seen in Figure 4.5, the back in Figure 4.6 and the full system with headphones in Figure 4.7

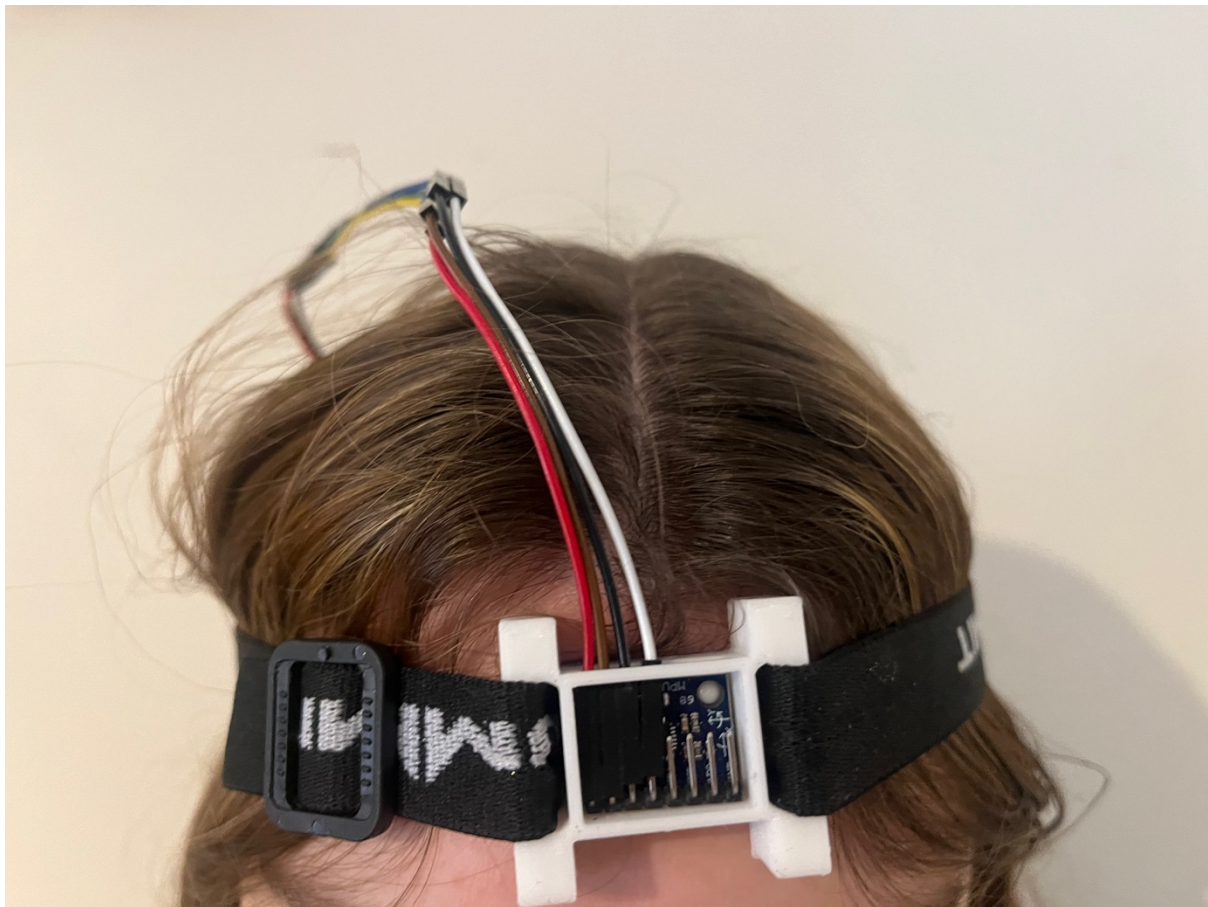


Figure 4.5 - View of the front of the device

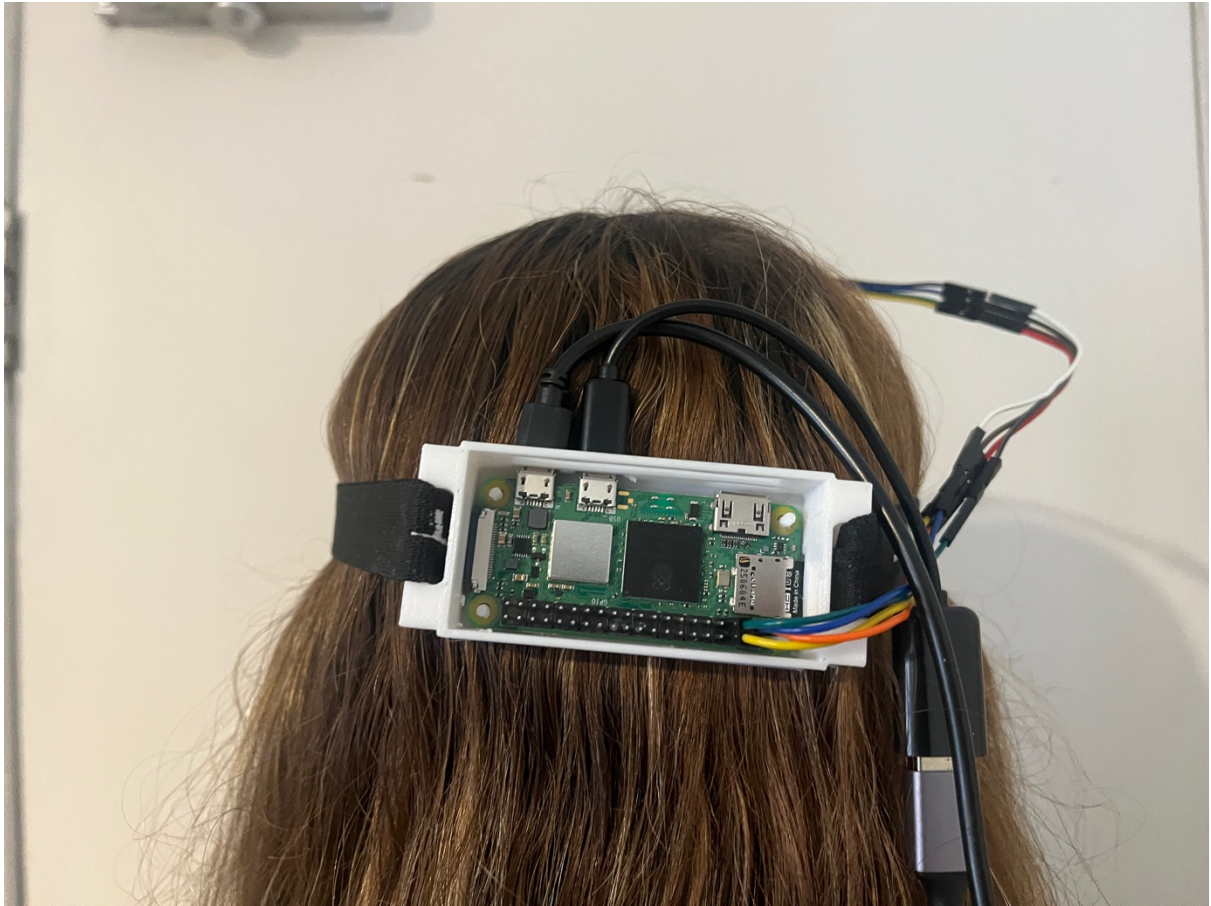


Figure 4.6 - View of the back of the device



Figure 4.7 - Full device with headphones

4.4 Software Stack

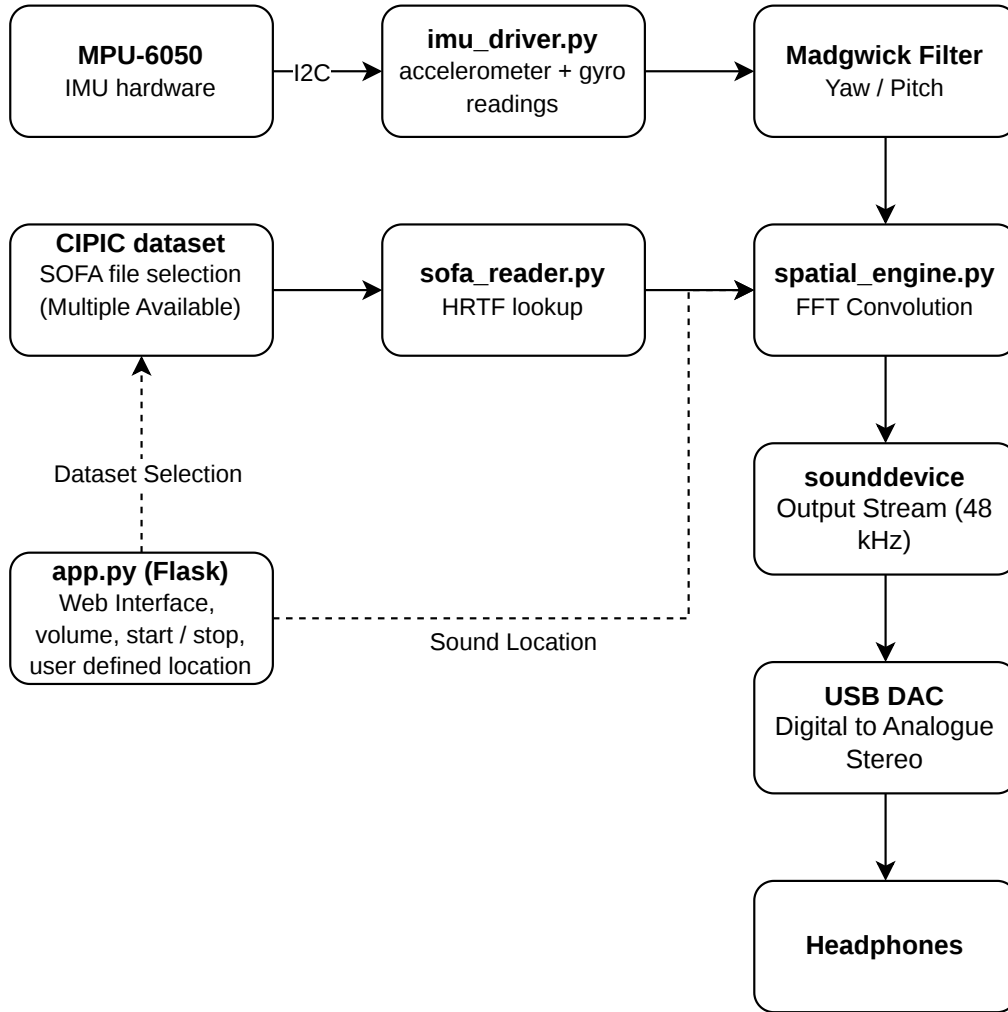


Figure 4.5 – Software pipeline architecture flowchart

The software pipeline is implemented entirely in Python 3 and split across four modules. The overall architecture is shown in Figure 4.5. Keeping the modules separate made development much easier as they could each be tested in isolation.

Imu_driver.py handles all the communication with the MPU-6050 and creates a stable orientation estimate to the rest of the pipeline. Sofa_reader.py loads the CIPIC dataset at startup and can provide the HRIR pairs on request. Spatial_engine.py is the core processing module as it brings the orientation data and HRIR lookup together into a continuous stereo audio stream. App.py provides a Flask web interface, providing control to the user without requiring a display or keyboard connected to the pi. Version control was used through git.

4.5 IMU Driver and Orientation Estimation

The MPU-6050 is initialised over I2C using smbus2, writing to the power management register to wake the device from sleep and configuring the accelerometer and gyroscope ranges. The raw output from the sensor is six 16-bit integers, which are three axes of acceleration and three of angular velocity, which are passed directly to the Madgwick filter.

As discussed in Section 3.4 the filter combines the accelerometer and gyroscope data to produce a drift corrected quaternion which is then converted to Euler angles for use by the spatial engine. The filter has a tuning parameter β which controls the weighting of the

accelerometer correction against the gyroscope integration. A higher value of β trusts the accelerometer more and corrects drift faster, whereas a value closer to zero causes the filter to rely much more on the gyroscope.

4.6 HRTF Convolution Engine

The CIPIC dataset is loaded on startup by `sofa_reader.py` using `pysofaconventions`, extracting the impulse response array, source position array and sampling rate as described in Section 3.3.

When the spatial engine requests a HRIR pair for a given orientation, the nearest neighbour lookup converts the target orientation and all stored measurement positions to three-dimensional unit vectors in Cartesian. The nearest match is determined by comparing the stored position with the minimum Euclidean distance to the target vector.

The selected left and right HRIRs are then convolved with the current audio frame using `scipy.signal.fftconvolve`. FFT based convolution was chosen over direct convolution because the HRIRs are 200 samples long at 48 kHz, making FFT convolution substantially faster.

4.7 Flask Web Interface and Calibration

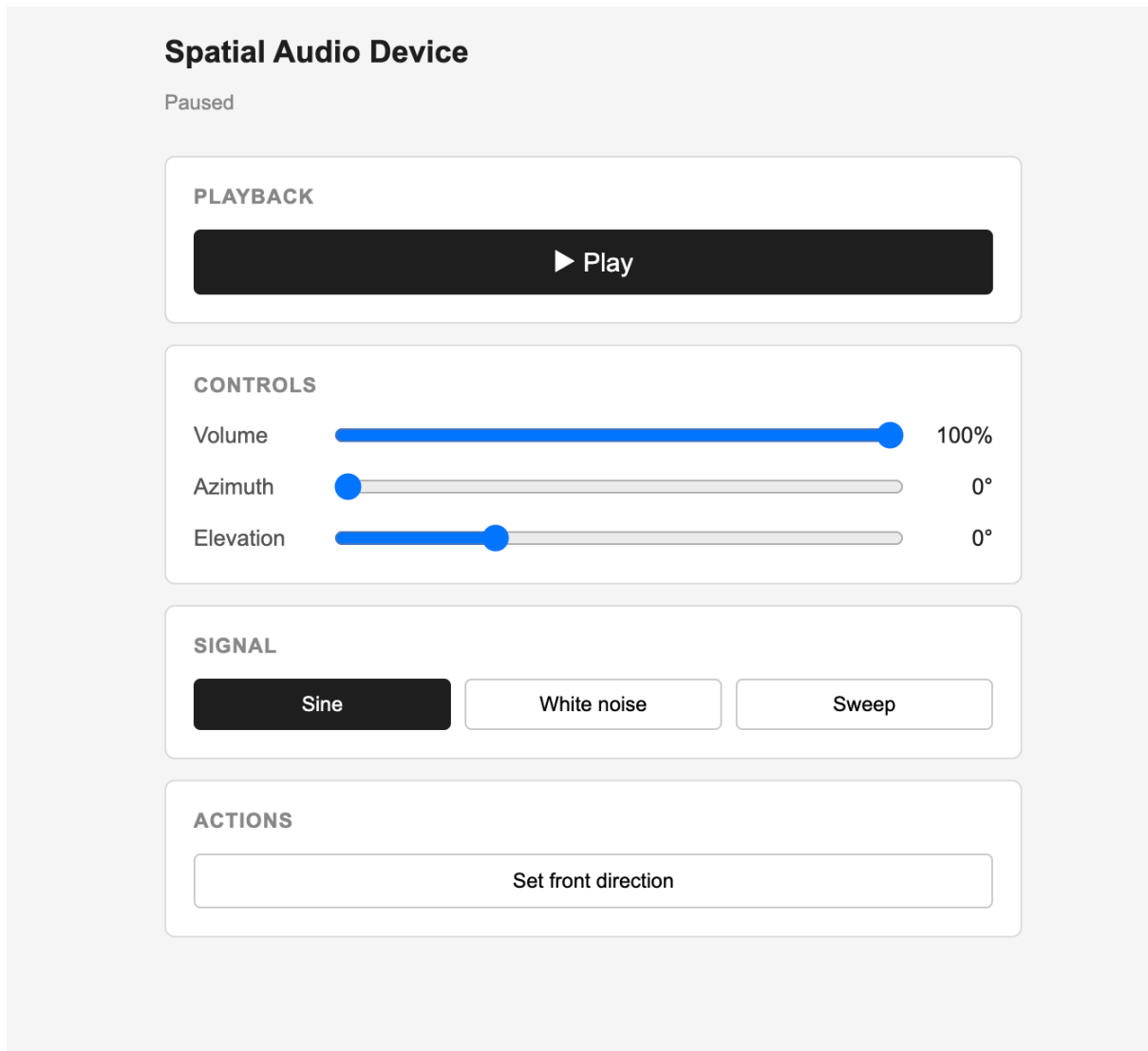


Figure 4.6: Flask web app interface

A Flask web application running on the Pi provides an interface reachable from any device on the same local network, as can be seen in figure 4.6. The interface provides controls for starting and stopping the audio engine, a volume slider and selecting the active HRTF dataset.

The interface also allows the user to specify the position of the virtual sound source by setting a target azimuth and elevation. This is the primary way navigation is achieved, as the user can place a sound source at a known location relative to the user, such as a doorway or object of interest, allowing the user can orient themselves towards it using the head tracked audio.

Rather than using runtime calibration of different parameters, the interface allows the user to switch between different HRTF subject measurements. As discussed in Section 3.3, localisation performance is heavily dependent on how closely a dataset matches the user's own head and ear geometry. By providing a selection of datasets, the user can find the most accurate one for their anatomy, which is more effective than adjusting parameters on a single dataset that may not suit the user's anatomy.

4.8 Summary

This chapter has described the complete hardware and software implementation of the audio reality navigation device. The physical prototype consists of a Raspberry Pi Zero 2 W, an MPU-6050 IMU and a USB audio adapter mounted on an elastic headband and wired with jumper wires. The software pipeline processes IMU data through a Madgwick filter to obtain stable head orientation, performs a nearest-neighbour HRTF lookup in the CIPIC dataset, convolves a tonal cue with the selected HRIRs using FFT-based convolution and outputs the result as a real-time stereo audio stream. A Flask web interface provides runtime control and calibration. The following chapter presents the testing and evaluation of the system, assessing spatialisation accuracy, pipeline latency and user response.

5. Solution Testing and Design Iterations

5.1 Test Strategy

Testing followed the same incremental approach as development where each subsystem was first tested in isolation before being integrated together. This made it straightforward to trace any issues back to their source and avoided faults in one component causing issues with other components.

5.2 IMU and Orientation Pipeline

Before integrating the IMU with the rest of the system, the sensor was tested on its own to check it was working and wired up correctly. Running `imu_driver.py` independently prints the live accelerometer and gyroscope readings to the terminal, which made it easy to check the output. With the headband sitting flat on a desk, the accelerometer z axis read around 1g as expected and all three gyroscope axes were close to 0°/s with only minor noise. This was enough to confirm the sensor was set up correctly and talking to the Pi over I2C.

Orientation accuracy was then assessed by running the full Madgwick filter pipeline and rotating the headband through known 90° increments. The printed yaw values at each position are shown in Table 5.1.

Physical Rotation (°)	Reported Yaw (°)	Error (°)
0	0	0
90	88.3	1.7
180	177.6	2.4
270	267.9	2.1

Table 5.1: Measured yaw accuracy across four orientations.

All Errors were under 2.5°, which is small enough to have no meaningful impact on HRTF selection given the angular spacing of measurements in the dataset.

Gyroscope drift was measured by leaving the device stationary for 5 minutes and logging the change in reported yaw over that period. The total drift observed was 10° , which equates to approximately $2^\circ/\text{minute}$. This is consistent with the expected behaviour of a low-cost gyroscope and is adequately corrected by the Madgwick filter during normal use, where the accelerometer continuously provides a gravity reference to anchor the orientation estimate.

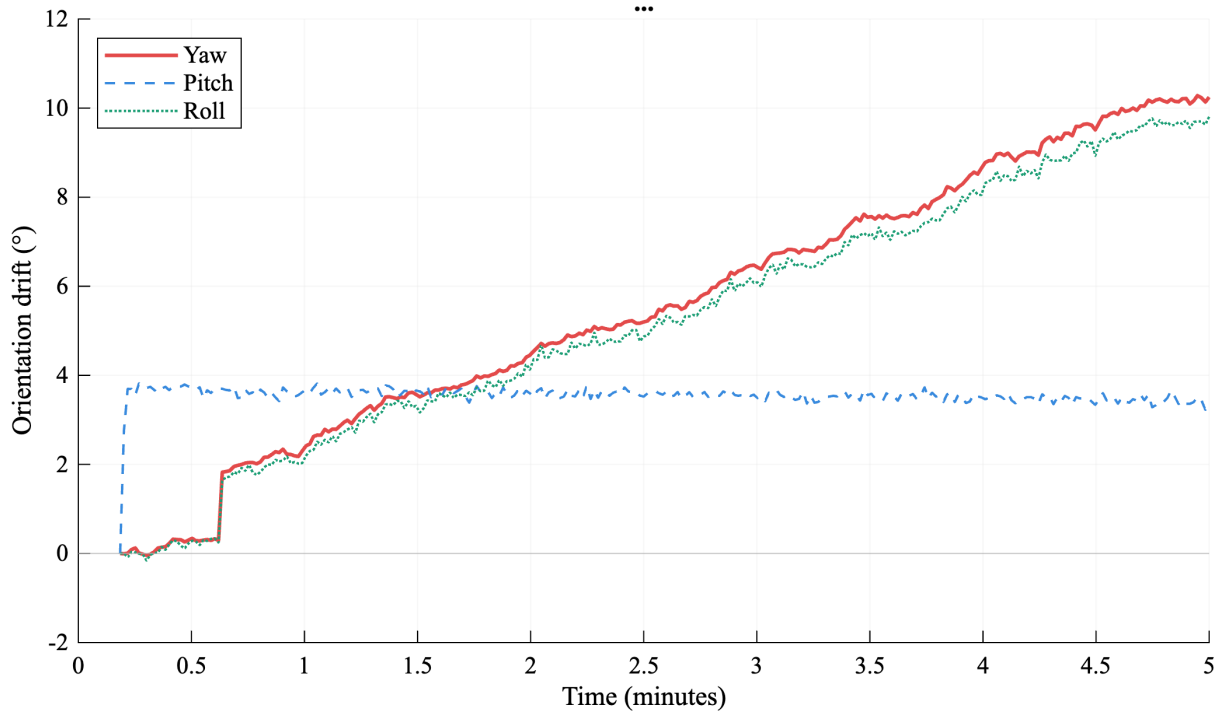


Figure 5.1: Orientation drift over five minutes with device stationary

The Madgwick filter convergence parameter β was tuned iteratively. An initial value of 0.001 was found to be far too low as it caused the filter to rely almost entirely on the gyroscope and accumulated drift rapidly. Increasing β to 0.033, which is the value recommended in the original Madgwick paper [19], produced a stable orientation that responded promptly to head movement without instability.

5.3 Audio Pipeline Latency

The latency of the device, which is the delay between a listener moving their head and the audio updating to match, is one of the most important parameters in a head tracked spatial audio system. If the latency is too high, the sound will appear to drift as the head moves, which can quickly become disorientating and confusing.

The theoretical minimum latency of the audio pipeline is determined by the block size used by the sounddevice output stream. With a block size of 256 samples at 48 kHz, each audio block represents approximately 5.3 ms of audio. The IMU sampling runs at 50 Hz, giving a maximum IMU update latency of 20 ms. The total theoretical pipeline latency is therefore approximately 25 ms, combining one IMU sample period with one audio block. For context, research suggests that head tracking latency below approximately 80 ms is generally unnoticeable to the average listener [23], which leaves a comfortable margin.

5.4 Spatialisation Accuracy

A listening test was conducted to evaluate how accurately the system conveyed the direction of a virtual sound source. White noise was selected as the test signal as its broad spectral content makes both interaural level differences and the spectral shaping applied by the HRTFs more perceptible than a pure tone. The test was conducted with the head stationary to avoid any head tracking error.

Virtual sources were presented at four azimuth positions in 45° increments, from 0° to 315° with elevation staying at 0°. For each position, the listener was asked to indicate the perceived direction of the sound from eight different options. Three listeners each completed five trials per position, giving 15 trials per position and 120 trials in total. The small participant sample of three listeners limits the validity of these results, a larger and more diverse group would improve accuracy. The results are shown in Table 5.2.

Target Azimuth (°)	Correct Responses (/15)	Correct Response Rate
0	11	73%
45	12	80%
90	14	93.3%
135	12	80%
180	9	60%
225	10	66.7%
270	13	86.7%
315	11	73.3%

Table 5.2: Spatialisation accuracy at eight azimuth positions.

Overall, 92 of 120 trials were correctly answered, giving an overall correct response rate of 76.7%. Localisation. Performance was highest at the positions of 90° and 270°, achieving a strong correct response rate of 93.3% and 86.7% respectively. Performance was weakest at 180° and 225°, where the correct response rates dropped to 60% and 66.7%. The incorrect responses were largely due to front and back reversals, where listeners were reporting a source at approximately 0° when the target was 180° and vice versa. This is a well-documented limitation of non-individualised HRTFs, as without pinnae measurements, the spectral cues that distinguish sources from front to back are often insufficient [15]. This also reflects Wightman and Kistlers findings [17], who showed that front back ambiguity is primarily resolved through head movement rather than static spectral cues, which they noticed when listeners who moved their head slightly showed noticeably fewer reversal errors.

5.5 Head-Tracked Spatial Audio

The fully integrated system was tested by wearing the headband and moving around, checking whether the sound source stayed fixed in space as the head rotated. This is the core behaviour of the device that needs to work, as if the audio doesn't track the head movement, then navigation won't be possible.

During normal use it worked well. Rotating left and right produced a clear, smooth and responsive shift in the perceived source direction, and during slow to moderate head movements the sound felt anchored in space, rather than attached to the head and moving with it. Latency between the head moving and the source position adjusting was not noticeable.

There was one issue observed during startup, which was that for a short period after starting up sources around 0° and 180° would occasionally jump to the wrong position before snapping back. This only happened for the first few seconds and was likely due to the Madgwick filter needing a short settling period after initialisation. During that time the filter is still converging and can produce abrupt output changes in the HRTF selection. In practice this was not a big issue, as waiting a few seconds for the filter to calibrate was sufficient, but a future fix would be to silence the audio until the filter has converged.

5.6 Design Iterations

There were several issues that came up during development that required debugging and redesigning, the most significant are described below.

Azimuth sign mismatch: When the system was first put together, rotating the head caused the sound source to spin quickly in the wrong direction, and would complete multiple full rotations for a small head movement. This traced back to two problems, the first of which was the Madgwick filter's β parameter was at 0.001, which was far too low and caused the filter to depend on the raw gyroscope data whilst ignoring the accelerometer entirely, causing rapid drift. The second problem was the azimuth value being added rather than negated before the HRTF lookup, which caused a mismatch between the filter's coordinate system and the SOFA dataset, causing the sound to rotate in the wrong direction. Changing this to a negative and increasing β to 0.033 fixed both issues.

IMU loop timing error: Another bug was found in the IMU thread in app.py. It had no sleep between iterations, meaning it was sampling the sensor at several hundred Hz, despite the filter being told it was running at 50Hz. As the filter was running at 50Hz to determine how much it weights each gyroscope reading, giving it data hundreds of times faster than expected caused it to over integrate, producing the same drifting issue as above. Adding a 20 ms sleep to the loop matched the two rates and resolved the issue.

Jumper wire contact reliability: I2C errors during early testing were traced to the jumper wire connections becoming unstable when the headband flexed. Routing the wires through the holes on the 3D printed enclosure reduced the mechanical stress enough to make the connections reliable. Replacing them with soldered connections in a future version would completely fix this issue.

6. Conclusion

The aim of this project was to build a low cost and wearable device capable of providing real time spatial audio guidance for visually impaired indoor navigation. Compared to the specifications set out in Section 2, the system performed well. Pipeline latency was measured at approximately 25 ms, well within the 80 ms target. Orientation accuracy remained within 2.5° across all tested positions, which is comfortably below the 5° threshold. Spatialisation accuracy across all eight azimuth positions averaged 76.7%, just exceeding the 75% target. The device was built for a total component cost of £55 which is marginally exceeding the £50 target, although this can be reduced by using a soldered battery pack instead of a shop bought power bank.

The most significant limitation is that the device only handles one virtual sound source at a fixed position set manually through the web interface. In a real navigation scenario, a user would be surrounded by multiple objects, none of which the device is currently aware of. Although this follows the original design, it means the device in its current form is more of a directional orientation tool than a full navigation system.

The use of non-personalised HRTFs from the CIPIC dataset introduced a secondary limitation. Front and back confusion was the most common error in the spatialisation test, making up most of the incorrect responses at 180° and 225° . This wasn't too surprising since non personalised HRTFs lack the individual ear measurements needed for reliable front and back discrimination as discussed in Section 3. Personalising the HRTFs to individual users turned out to be much more complex than initially anticipated and was not achieved within this project.

The absence of a magnetometer means the system has no absolute heading reference. Yaw drift accumulates over time and requires the user to periodically recalibrate by returning to a known reference orientation. Adding a magnetometer in a future version would provide a stable heading and eliminate this limitation entirely.

The most impactful direction for future work would be to integrate an object detection system such as a depth camera or ultrasonic array to automatically identify objects in the room and feed their positions to the spatial engine. This would transform the device from a single target orientation device into a more complete indoor navigation system.

References

- [1] 'World report on vision'. Accessed: Mar. 18, 2026. [Online]. Available: <https://www.who.int/publications/i/item/9789241516570>
- [2] J. Zaba, 'CHILDREN'S VISION CARE IN THE 21ST CENTURY & ITS IMPACT ON EDUCATION, LITERACY, SOCIAL ISSUES, & THE WORKPLACE: A CALL TO ACTION', *Journal of Behavioral Optometry*, vol. 22, p. 39, Nov. 2011.

- [3] W. R. Wiener, R. L. Welsh, and B. B. Blasch, Eds, *Foundations of orientation and mobility*, 3rd ed. New York: AFB Press, 2010.
- [4] R. R. A. Bourne *et al.*, 'Magnitude, temporal trends, and projections of the global prevalence of blindness and distance and near vision impairment: a systematic review and meta-analysis', *The Lancet Global Health*, vol. 5, no. 9, pp. e888–e897, Sep. 2017, doi: 10.1016/S2214-109X(17)30293-0.
- [5] J. M. Loomis, R. G. Golledge, and R. L. Klatzky, 'Navigation System for the Blind: Auditory Display Modes and Guidance', *Presence*, vol. 7, no. 2, pp. 193–203, Apr. 1998, doi: 10.1162/105474698565677.
- [6] W. Elmannai and K. Elleithy, 'Sensor-Based Assistive Devices for Visually-Impaired People: Current Status, Challenges, and Future Directions', *Sensors*, vol. 17, no. 3, p. 565, Mar. 2017, doi: 10.3390/s17030565.
- [7] 'WeWALK', WeWALK Smart Cane. Accessed: Mar. 18, 2026. [Online]. Available: <https://wewalk.io/en/>
- [8] R. N. Kandalan and K. Namuduri, 'Techniques for Constructing Indoor Navigation Systems for the Visually Impaired: A Review', *IEEE Trans. Human-Mach. Syst.*, vol. 50, no. 6, pp. 492–506, Dec. 2020, doi: 10.1109/THMS.2020.3016051.
- [9] A. T. Akın and Ç. Cömert, 'The development of an augmented reality audio application for visually impaired persons', *Multimed Tools Appl*, vol. 82, no. 11, pp. 17493–17512, May 2023, doi: 10.1007/s11042-022-14134-x.
- [10] J. Blauert, *Spatial Hearing: The Psychophysics of Human Sound Localization*. MIT Press, 1997.
- [11] D. Hammershoi and H. Møller, 'Binaural Technique — Basic Methods for Recording, Synthesis, and Reproduction', in *Communication Acoustics*, J. Blauert, Ed., Berlin/Heidelberg: Springer-Verlag, 2005, pp. 223–254. doi: 10.1007/3-540-27437-5_9.
- [12] V. R. Algazi, R. O. Duda, D. M. Thompson, and C. Avendano, 'The CIPIC HRTF database', in *Proceedings of the 2001 IEEE Workshop on the Applications of Signal Processing to Audio and Acoustics (Cat. No. 01TH8575)*, New Platz, NY, USA: IEEE, 2001, pp. 99–102. doi: 10.1109/ASPAA.2001.969552.
- [13] Piotr Majdak, Yukio Iwaya, Thibaut Carpentier, Rozenn Nicol, Matthieu Parmentier, Agnieszka Roginska, Yôiti Suzuki, Kanji Watanabe, Hagen Wierstorf, Harald Ziegelwanger, Markus Noisternig, 'Spatially Oriented Format for Acoustics: A Data Exchange Format Representing Head-Related Transfer Functions', presented at the Audio Engineering Society 134th Convention, Rome, Italy, May 2013. [Online]. Available: https://hagenw.github.io/pubs/majdak_et_al-2013-spatially_oriented_format_for_acoustics.pdf
- [14] F. Ribeiro, D. Florencio, P. A. Chou, and Z. Zhang, 'Auditory augmented reality: Object sonification for the visually impaired', in *2012 IEEE 14th International Workshop on Multimedia Signal Processing (MMSP)*, Banff, AB, Canada: IEEE, Sep. 2012, pp. 319–324. doi: 10.1109/MMSP.2012.6343462.
- [15] E. M. Wenzel, M. Arruda, D. J. Kistler, and F. L. Wightman, 'Localization using nonindividualized head-related transfer functions', *The Journal of the Acoustical Society of America*, vol. 94, no. 1, pp. 111–123, Jul. 1993, doi: 10.1121/1.407089.
- [16] V. Planinec, J. Reijniers, M. Horvat, H. Peremans, and K. Jambrošić, 'The Accuracy of Dynamic Sound Source Localization and Recognition Ability of Individual Head-Related Transfer Functions in Binaural Audio Systems with Head Tracking', *Applied Sciences*, vol. 13, no. 9, p. 5254, Apr. 2023, doi: 10.3390/app13095254.
- [17] F. L. Wightman and D. J. Kistler, 'Resolution of front–back ambiguity in spatial hearing by listener and source movement', *The Journal of the Acoustical Society of America*, vol. 105, no. 5, pp. 2841–2853, May 1999, doi: 10.1121/1.426899.
- [18] InvenSense, 'MPU-6000 and MPU-6050 Product Specification'. TDK InvenSense, Aug. 19, 2013. [Online]. Available: <https://xonstorage.z8.web.core.windows.net/pdf/3394407link.pdf>
- [19] S. O. H. Madgwick, A. J. L. Harrison, and R. Vaidyanathan, 'Estimation of IMU and MARG orientation using a gradient descent algorithm', in *2011 IEEE International Conference on Rehabilitation Robotics*, Zurich: IEEE, Jun. 2011, pp. 1–7. doi: 10.1109/ICORR.2011.5975346.
- [20] Raspberry Pi Foundation, 'Raspberry Pi 3 Model B Product Datasheet'. RS Components, 2016. [Online]. Available: https://us.rs-online.com/m/d/4252b1ecd92888dbb9d8a39b536e7bf2.pdf?srsltid=AfmBOorG_gCOYGUm1NddO0SxkOtOrufCNqJUG3A46h9nguG5FOpg7I
- [21] Raspberry Pi Ltd, 'Raspberry Pi Zero 2 W Product Brief'. Raspberry Pi Ltd, Apr. 2024. [Online]. Available: <https://pip-assets.raspberrypi.com/categories/584-raspberry-pi-zero-2-w/documents/RP-008359-DS-1-raspberry-pi-zero-2-w-product-brief.pdf?disposition=inline>
- [22] M. Frigo and S. G. Johnson, 'The Design and Implementation of FFTW3', *Proc. IEEE*, vol. 93, no. 2, pp. 216–231, Feb. 2005, doi: 10.1109/JPROC.2004.840301.
- [23] D. S. Bunghart, B. D. Simpson, and A. J. Kordik, 'The Detectability of Headtracker Latency in Virtual Audio Displays', presented at the International Conference on Auditory Display, Limerick, Ireland, 2005. [Online]. Available: <https://www.icad.org/Proceedings/2005/BrungartSimpson2005.pdf>

Appendix A – Code Listings

A.1 – app.py

```
from flask import Flask, request, jsonify, render_template_string
from spatial_engine import SpatialEngine
from calibration import CalibrationProfile, list_sofa_files, PROFILE_PATH

import threading
import numpy as np
import ahrs
import time
from imu_driver import IMUDriver

engine = SpatialEngine()
engine.start()

heading_reset = threading.Event()
fixed_point_mode = threading.Event()
calibration_active = threading.Event()

fixed_target_az = 0.0
fixed_target_lock = threading.Lock()
fixed_yaw_snapshot = 0.0
fixed_snapshot_lock = threading.Lock()

def imu_loop():
    imu = IMUDriver()
    imu.start()

    # collect 200 samples with the device stationary to
    # estimate gyro bias as this drift would otherwise corrupt heading over time
    print("Calibrating gyro offset – keep device still...")
    offset_samples = []
    for _ in range(200):
        t0 = time.time()
        d = imu.read()
        offset_samples.append([d['gyro']['x'], d['gyro']['y'], d['gyro']['z']])
        time.sleep(max(0, 0.02 - (time.time() - t0)))
    offset_samples = np.array(offset_samples)
    gx_off = float(np.mean(offset_samples[:, 0]))
    gy_off = float(np.mean(offset_samples[:, 1]))
    gz_off = float(np.mean(offset_samples[:, 2]))
    print(f"Gyro offsets – x:{gx_off:.3f} y:{gy_off:.3f} z:{gz_off:.3f}")

    # madgwick filter runs at 50 Hz and beta=0.033 is a conservative value that
    # keeps the heading stable
    madgwick = ahrs.filters.Madgwick(frequency=50.0, beta=0.033)
    Q = np.array([1.0, 0.0, 0.0, 0.0])

    last_yaw = None
    current_az = 0
    was_fixed = False
    SENSITIVITY = 0.3 # scales how much head rotation moves the audio
    MOUNT_OFFSET = 180.0 # the IMU is mounted facing backward on the headband

    while True:
        t0 = time.time()
        try:
            data = imu.read()
            acc = np.array([data['accel']['x'], data['accel']['y'],
data['accel']['z']])
            gyr = np.radians([
                data['gyro']['x'] - gx_off,
                data['gyro']['y'] - gy_off,
                data['gyro']['z'] - gz_off,
```

```

    ])
    Q = madgwick.updateIMU(Q, gyr=gyr, acc=acc)
    angles = ahrs.common.orientation.q2euler(Q)
    yaw = np.degrees(angles[2]) % 360.0
    pitch = np.degrees(angles[1])

    if last_yaw is not None and not calibration_active.is_set():
        if fixed_point_mode.is_set():
            # fixed-point mode: audio source stays locked in world space
            # as the user rotates their head
            if not was_fixed:
                with fixed_snapshot_lock:
                    fixed_yaw_snapshot = yaw
                was_fixed = True
            with fixed_snapshot_lock:
                snapshot = fixed_yaw_snapshot
            with fixed_target_lock:
                target = fixed_target_az
            head_delta = (yaw - snapshot + 360.0) % 360.0
            if head_delta > 180.0:
                head_delta -= 360.0
            engine.set_azimuth((target - head_delta) % 360.0)
        else:
            was_fixed = False
            if heading_reset.is_set():
                current_az = 0
                heading_reset.clear()
            delta = yaw - last_yaw
            if delta > 180:
                delta -= 360
            elif delta < -180:
                delta += 360
            # subtract delta so turning right moves the source leftward
            current_az = (current_az - delta * SENSITIVITY) % 360.0
            output_az = (current_az + MOUNT_OFFSET) % 360.0
            engine.set_azimuth(output_az)
            engine.set_elevation(pitch)

    last_yaw = yaw

except Exception as e:
    print(f"IMU error: {e}")
    break

elapsed = time.time() - t0
time.sleep(max(0, 0.02 - elapsed))

imu_thread = threading.Thread(target=imu_loop, daemon=True)
imu_thread.start()

app = Flask(__name__)

@app.route("/")
def index():
    return render_template_string("index.html")

@app.route("/api/state", methods=["GET"])
def api_state_get():
    return jsonify(engine.get_state())

@app.route("/api/state", methods=["POST"])
def api_state_post():
    global fixed_target_az
    data = request.get_json(force=True, silent=True) or {}

```

```

        if "paused" in data: engine.set_paused(data["paused"])
        if "azimuth" in data:
            if fixed_point_mode.is_set():
                with fixed_target_lock:
                    fixed_target_az = float(data["azimuth"])
        if "elevation" in data: engine.set_elevation(data["elevation"])
        if "signal" in data: engine.set_signal(data["signal"])
        return jsonify(engine.get_state())

@app.route("/api/toggle", methods=["POST"])
def api_toggle():
    engine.toggle_paused()
    return jsonify(engine.get_state())

@app.route("/api/volume", methods=["POST"])
def api_volume():
    data = request.get_json(force=True, silent=True) or {}
    engine.set_volume(float(data.get("volume", 1.0)))
    return jsonify(engine.get_state())

@app.route("/api/fixed_point", methods=["POST"])
def api_fixed_point():
    data = request.get_json(force=True, silent=True) or {}
    if data.get("enabled"):
        fixed_point_mode.set()
    else:
        fixed_point_mode.clear()
    return jsonify({"ok": True})

@app.route("/api/calibrate/heading", methods=["POST"])
def api_calibrate_heading():
    heading_reset.set()
    return jsonify({"ok": True})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=False)

```

A.2 – spatial_engine.py

```

import atexit
import logging
import math
import threading
import numpy as np
import sounddevice as sd
from numpy.fft import rfft, irfft
from sofa_reader import HRTFDatabase

log = logging.getLogger(__name__)

def make_pink_block(white, state):
    out = np.empty(len(white), dtype=np.float64)
    b = state
    for i, w in enumerate(white):
        b[0] = 0.99886 * b[0] + w * 0.0555179

```

```

b[1] = 0.99332 * b[1] + w * 0.0750759
b[2] = 0.96900 * b[2] + w * 0.1538520
b[3] = 0.86650 * b[3] + w * 0.3104856
b[4] = 0.55000 * b[4] + w * 0.5329522
b[5] = -0.7616 * b[5] - w * 0.0168980
out[i] = (b[0]+b[1]+b[2]+b[3]+b[4]+b[5]+b[6] + w*0.5362) * 0.11
b[6] = w * 0.115926
return out.astype(np.float32)

```

```

class SpatialEngine:
    #real-time stereo spatial audio engine using HRTF convolution.

    SIGNALS = frozenset({"sine", "noise", "pink", "sweep"})

    def __init__(
        self,
        fs=48000,
        tone_hz=1000,
        blocksize=256,
        sofa_path="data/hrtf_nh2.sofa",
    ):
        self.fs = int(fs)
        self.blocksize = int(blocksize)
        self.tone_hz = float(tone_hz)
        self.volume = 1.0

        # lock protects all state that the IMU thread and flask routes can touch
        self.lock = threading.Lock()
        self.paused = True
        self.azimuth = 0.0
        self.elevation = 0.0
        self.signal = "sine"
        self.heading_offset = 0.0

        self.phase = 0.0
        self.phase_inc = 2.0 * math.pi * self.tone_hz / self.fs
        self.rng = np.random.default_rng(0)
        self.pink_state = np.zeros(7, dtype=np.float64)
        self.sweep_pos = 0
        self.callback_count = 0

        self.hrtf_db = self.load_db(sofa_path)
        self.build_fft_table()

        self.stream = None
        atexit.register(self.stop)

    def load_db(self, sofa_path):
        db = HRTFDatabase(sofa_path)
        if db.fs != self.fs:
            db.close()
            raise ValueError(
                f"HRTF fs={db.fs} does not match engine fs={self.fs}"
            )
        return db

    def build_fft_table(self):
        #pre-compute rfft of every HRIR pair so the callback just does a multiply.
        hrir_len = self.hrtf_db.ir.shape[2]
        raw = self.blocksize + hrir_len - 1
        # round up to next power of two for efficient FFT
        self.fft_len = 1 << (raw - 1).bit_length()

        n_pos = self.hrtf_db.ir.shape[0]
        self.hrir_ffts = []

```

```

    for i in range(n_pos):
        hL = self.hrtf_db.ir[i, 0, :].astype(np.float32)
        hR = self.hrtf_db.ir[i, 1, :].astype(np.float32)
        self.hrir_ffts.append((rfft(hL, n=self.fft_len),
                               rfft(hR, n=self.fft_len)))

    tail = hrir_len - 1
    self.overlap_L = np.zeros(tail, dtype=np.float32)
    self.overlap_R = np.zeros(tail, dtype=np.float32)
    self.tail = tail

def set_azimuth(self, az_deg):
    with self.lock:
        self.azimuth = float(az_deg) % 360.0

def set_elevation(self, el_deg):
    with self.lock:
        self.elevation = float(el_deg)

def set_signal(self, sig):
    sig = sig.lower().strip()
    if sig not in self.SIGNALS:
        sig = "noise"
    with self.lock:
        self.signal = sig

def set_paused(self, paused):
    with self.lock:
        self.paused = bool(paused)

def toggle_paused(self):
    with self.lock:
        self.paused = not self.paused
    return self.paused

def set_volume(self, vol):
    with self.lock:
        self.volume = float(np.clip(vol, 0.0, 1.0))

def set_heading_offset(self):
    with self.lock:
        self.heading_offset = self.azimuth

def get_state(self):
    with self.lock:
        return {
            "azimuth": self.azimuth,
            "elevation": self.elevation,
            "signal": self.signal,
            "paused": self.paused,
            "volume": self.volume,
        }

# signal generators

def gen_sine(self, frames):
    t = np.arange(frames, dtype=np.float64)
    block = np.sin(self.phase + t * self.phase_inc).astype(np.float32)
    self.phase = (self.phase + frames * self.phase_inc) % (2.0 * math.pi)
    return block

def gen_noise(self, frames):
    return (self.rng.standard_normal(frames) * 0.12).astype(np.float32)

def gen_pink(self, frames):
    return make_pink_block(self.rng.standard_normal(frames), self.pink_state)

```

```

def gen_sweep(self, frames):
    period = self.fs
    f_lo, f_hi = 200.0, 16000.0
    idx = (self.sweep_pos + np.arange(frames)) % period
    self.sweep_pos = int((self.sweep_pos + frames) % period)
    f = f_lo * (f_hi / f_lo) ** (idx / period)
    block = np.sin(np.cumsum(2.0 * math.pi * f / self.fs)).astype(np.float32) *
0.25
    return block

# audio callback

def callback(self, outdata, frames, time_info, status):
    if status:
        log.warning("sounddevice status: %s", status)

    with self.lock:
        az = self.azimuth
        el = self.elevation
        sig = self.signal
        paused = self.paused

    if paused:
        outdata[:] = 0
        return

    self.callback_count += 1

    if sig == "sine":
        mono = self.gen_sine(frames)
    elif sig == "pink":
        mono = self.gen_pink(frames)
    elif sig == "sweep":
        mono = self.gen_sweep(frames)
    else:
        mono = self.gen_noise(frames)

    # overlap add convolution with the nearest HRIR pair
    _, _, idx, _, _ = self.hrtf_db.get_nearest_hrir(az, el)
    hL_fft, hR_fft = self.hrir_ffts[idx]

    mono_fft = rfft(mono, n=self.fft_len)
    conv_L = irfft(mono_fft * hL_fft)[:frames + self.tail].astype(np.float32)
    conv_R = irfft(mono_fft * hR_fft)[:frames + self.tail].astype(np.float32)

    if self.tail > 0:
        conv_L[:self.tail] += self.overlap_L
        conv_R[:self.tail] += self.overlap_R
        self.overlap_L = conv_L[frames:frames + self.tail].copy()
        self.overlap_R = conv_R[frames:frames + self.tail].copy()

    out = np.stack([conv_L[:frames], conv_R[:frames]], axis=1)
    outdata[:] = np.clip(out * self.volume, -0.9, 0.9)

def start(self):
    if self.stream is not None:
        return
    self.stream = sd.OutputStream(
        device=0,
        samplerate=self.fs,
        channels=2,
        blocksize=self.blocksize,
        dtype="float32",
        callback=self.callback,

```

```

    )
    self.stream.start()

def stop(self):
    if self.stream is not None:
        self.stream.stop()
        self.stream.close()
        self.stream = None
    if self.hrtf_db is not None:
        self.hrtf_db.close()

def __enter__(self):
    self.start()
    return self

def __exit__(self, *args):
    self.stop()

```

A.3 – sofa_reader.py

```

import math
import numpy as np
from pysofaconventions import SOFAFile

class HRTFDatabase:

    #loads an HRTF SOFA file and provides fast nearest HRIR lookup

    def __init__(self, sofa_path):
        self.sofa_path = sofa_path
        self.sofa = None

        sofa = SOFAFile(sofa_path, "r")
        try:
            ir = sofa.getDataIR()
            positions = sofa.getVariableValue("SourcePosition")
            fs = int(sofa.getSamplingRate())
        except Exception:
            sofa.close()
            raise

        if ir.ndim != 3 or ir.shape[1] < 2:
            sofa.close()

        if positions.ndim != 2 or positions.shape[1] < 2:
            sofa.close()

        self.sofa = sofa
        self.fs = fs

        # store as float32 to keep memory down
        self.ir = ir.astype(np.float32)
        az_deg = positions[:, 0]
        el_deg = positions[:, 1]

        self.az_wrapped = np.mod(az_deg, 360.0)
        self.el = el_deg.copy()

        # pre compute unit vectors for every measurement direction so that
        # nearest-neighbour lookup is just a single dot product argmax
        az_r = np.deg2rad(az_deg)
        el_r = np.deg2rad(el_deg)

```

```

        cos_el = np.cos(el_r)
        self.unit_vecs = np.column_stack([
            cos_el * np.sin(az_r),
            cos_el * np.cos(az_r),
            np.sin(el_r),
        ]).astype(np.float32)

    def get_nearest_hrir(self, target_azimuth, target_elevation):
        az_r = math.radians(float(target_azimuth) % 360.0)
        el_r = math.radians(float(target_elevation))
        cos_el = math.cos(el_r)

        q = np.array([
            cos_el * math.sin(az_r),
            cos_el * math.cos(az_r),
            math.sin(el_r),
        ], dtype=np.float32)

        dots = self.unit_vecs @ q
        idx = int(np.argmax(dots))

        hL = self.ir[idx, 0, :]
        hR = self.ir[idx, 1, :]
        return hL, hR, idx, float(self.az_wrapped[idx]), float(self.el[idx])

    def close(self):
        if self.sofa is not None:
            self.sofa.close()
            self.sofa = None

    def __enter__(self):
        return self

    def __exit__(self, *args):
        self.close()

```

A.4 – imu_driver.py

```

import time
import smbus2

# default I2C address when AD0 pin is left unconnected
MPU6050_ADDR = 0x68

# register addresses
PWR_MGMT_1 = 0x6B # power management – write 0x00 to wake from sleep
ACCEL_XOUT_H = 0x3B # first of 6 accelerometer registers
GYRO_XOUT_H = 0x43 # first of 6 gyroscope registers
ACCEL_CONFIG = 0x1C # accelerometer full-scale range config
GYRO_CONFIG = 0x1B # gyroscope full-scale range config

# full range settings
ACCEL_FS = 0x00 # ±2g
GYRO_FS = 0x08 # ±500/s

ACCEL_SCALE = 16384.0
GYRO_SCALE = 65.5

def s16(high, low):
    #combine two bytes into a signed 16-bit integer
    val = (high << 8) | low
    return val - 65536 if val >= 32768 else val

```

```

class IMUDriver:

    def __init__(self, i2c_bus=1, address=MPU6050_ADDR):
        self.bus_num = i2c_bus
        self.address = address
        self.bus = None

    def start(self):
        #open the I2C bus and wake the MPU-6050 from sleep
        self.bus = smbus2.SMBus(self.bus_num)
        # register 0x6B = 0x00 clears the sleep bit
        self.bus.write_byte_data(self.address, PWR_MGMT_1, 0x00)
        time.sleep(0.1) # let the sensor settle before we start reading
        self.bus.write_byte_data(self.address, ACCEL_CONFIG, ACCEL_FS)
        self.bus.write_byte_data(self.address, GYRO_CONFIG, GYRO_FS)

    def stop(self):
        #close the I2C bus
        if self.bus is not None:
            self.bus.close()
            self.bus = None

    def read(self):

        if self.bus is None:
            raise RuntimeError("IMUDriver not started - call start() first")

        # 6 bytes covers X, Y, Z high+low pairs
        raw_a = self.bus.read_i2c_block_data(self.address, ACCEL_XOUT_H, 6)
        ax = s16(raw_a[0], raw_a[1]) / ACCEL_SCALE
        ay = s16(raw_a[2], raw_a[3]) / ACCEL_SCALE
        az = s16(raw_a[4], raw_a[5]) / ACCEL_SCALE

        raw_g = self.bus.read_i2c_block_data(self.address, GYRO_XOUT_H, 6)
        gx = s16(raw_g[0], raw_g[1]) / GYRO_SCALE
        gy = s16(raw_g[2], raw_g[3]) / GYRO_SCALE
        gz = s16(raw_g[4], raw_g[5]) / GYRO_SCALE

        return {
            "accel": {"x": ax, "y": ay, "z": az},
            "gyro": {"x": gx, "y": gy, "z": gz},
        }

    def __enter__(self):
        self.start()
        return self

    def __exit__(self, *args):
        self.stop()

```